



UNIVERSIDAD AUTÓNOMA DE SINALOA

FACULTAD DE INFORMÁTICA CULIACÁN



MANUAL DE PRÁCTICAS DE MONGODB

COLECCIÓN, CONSULTA, ACTUALIZACIÓN Y
ELIMINACIÓN DE DOCUMENTOS CON MONGODB



ASIGNATURA:
Bases de Datos
Emergentes



PROGRAMA EDUCATIVO:
Licenciatura en Ingeniería
en Ciencia de Datos



RESPONSABLE:
Dra. Lidia Yadira
Pérez Aguilar



FECHA:
agosto 2026



DATOS • CONOCIMIENTO • DECISIONES

Tecnología que transforma información en valor



PRÁCTICA 3. Operadores, arrays y condiciones combinadas

Objetivo

Resolver consultas sobre documentos MongoDB mediante operadores de comparación, lógicos, existencia y arrays, construyendo filtros progresivamente más complejos sobre la colección productos.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia de las consultas ejecutadas y analizar los resultados obtenidos.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Preparar el entorno de trabajo

- **Poner en uso nuestra base de datos**

```
use ventas_online
```

- **Contar los documentos**

```
db.productos.countDocuments()
```

Resultado esperado: 50 documentos.

1. Operadores de comparación

- `$eq` → igual a.
- `$ne` → diferente de.
- `$gt` → mayor a.
- `$gte` → mayor o igual que.
- `$lt` → menor a.
- `$lte` → menor o igual que.
- `$in` → coincide con alguno de los valores indicados.
- `$nin` → no coincide con ninguno de los valores indicados.

1.1 Buscar productos cuyo precio sea exactamente igual a 1450

```
db.productos.find({  
  precio: { $eq: 1450 }  
})
```

También puede escribirse de forma simplificada:



```
db.productos.find({
  precio: 1450
})
```

1.2 Buscar productos cuya categoría sea diferente de "Audio"

```
db.productos.find({
  categoria: { $ne: "Audio" }
})
```

1.3 Buscar productos cuyo precio sea mayor a \$1,000

```
db.productos.find({
  precio: { $gt: 1000 }
})
```

1.4 Buscar productos cuyo precio sea mayor o igual a \$1,000

```
db.productos.find({
  precio: { $gte: 1000 }
})
```

1.5 Buscar productos cuyo stock sea menor a 20

```
db.productos.find({
  stock: { $lt: 20 }
})
```

1.6 Buscar productos cuyo precio sea menor o igual a \$1,500

```
db.productos.find({
  precio: { $lte: 1500 }
})
```

1.7 Buscar productos cuyo precio esté entre \$1,000 y \$10,000

```
db.productos.find({
  precio: {
    $gte: 1000,
    $lte: 10000
  }
})
```

1.8 Buscar productos cuyo stock esté entre 15 y 40 unidades, incluyendo ambos valores.

```
db.productos.find({
  stock: {
    $gte: 15,
    $lte: 40
  }
})
```

1.9 Buscar productos cuyo precio sea mayor a \$500 pero menor a \$2,000

```
db.productos.find({
  precio: {
    $gt: 500,
    $lt: 2000
  }
})
```

1.10 Buscar productos pertenecientes a las categorías "Audio" o "Computo"

```
db.productos.find({
  categoria: {
    $in: ["Audio", "Computo"]
  }
})
```



1.11 Buscar productos que no pertenezcan a las categorías "Audio" ni "Computo"

```
db.productos.find({
  categoria: {
    $nin: ["Audio", "Computo"]
  }
})
```

2. Operadores lógicos

- \$and: deben cumplirse todas las condiciones.
- \$or: debe cumplirse al menos una.
- \$nor: no debe cumplirse ninguna.
- \$not: niega una condición determinada.

2.1 Buscar productos cuyo precio sea menor a \$5,000 y cuyo stock sea mayor a 20 unidades

```
db.productos.find({
  $and: [
    {precio:{$lt:5000}},
    {stock:{$gt:20}}
  ]
})
```

2.2 Buscar productos que pertenezcan a la categoría "Audio" o cuyo stock sea menor a 15 unidades

```
db.productos.find({
  $or: [
    {categoria:"Audio"},
    {stock:{$lt:15}}
  ]
})
```

2.3 Buscar productos que no pertenezcan a la categoría "Audio" ni tengan un precio mayor a \$5,000

```
db.productos.find({
  $nor: [
    {categoria:"Audio"},
    {precio:{$gt:5000}}
  ]
})
```

2.4 Buscar productos cuyo precio no sea mayor a \$5,000

```
db.productos.find({
  precio: {
    $not: {$gt:5000}
  }
})
```



3. Operadores de existencia \$exists

Esta parte es muy importante porque nuestros productos no tienen todos exactamente la misma estructura:

3.1 Buscar productos que tengan el campo especificaciones

```
db.productos.find({
  especificaciones:{$exists:true}
})
```

3.2 Buscar productos que no tengan el campo especificaciones

```
db.productos.find({
  especificaciones:{$exists:false}
})
```

3.3 Buscar productos que tengan definido el campo bluetooth

```
db.productos.find({
  bluetooth:{$exists:true}
})
```

Nota: La consulta no está preguntando si bluetooth es true. Está preguntando si el campo bluetooth existe en el documento

4. Operadores sobre arrays

Ya tenemos etiquetas, por lo que podemos aprovecharla:

4.1 Buscar productos cuyo array etiquetas contenga el valor "gaming"

```
db.productos.find({
  etiquetas:"gaming"
})
```

4.2 Buscar productos cuyo array etiquetas contenga simultáneamente los valores "audio" y "promocion"

```
db.productos.find({
  etiquetas:{
    $all:["audio","promocion"]
  }
})
```

Nota: El operador \$all exige que el array contenga todos los valores indicados, aunque pueda contener además otras etiquetas.

4.3 Buscar productos cuyo array etiquetas contenga exactamente dos elementos

```
db.productos.find({
  etiquetas:{$size:2}
})
```

Nota: El operador \$size no busca un contenido específico, sino la cantidad de elementos almacenados en el array.



4.4 Buscar productos que tengan el campo etiquetas

```
db.productos.find({
  etiquetas:{$exists:true}
})
```

Y podríamos contrastarla con:

4.5 Buscar productos que no tengan el campo etiquetas

```
db.productos.find({
  etiquetas:{$exists:false}
})
```

5. Condiciones combinadas

5.1 Buscar productos de categoría "Audio" cuyo precio sea mayor a \$1,000

```
db.productos.find({
  categoria:"Audio",
  precio:{$gt:1000}
})
```

5.2 Buscar productos de categoría "Computo" cuyo stock sea menor a 20

```
db.productos.find({
  categoria:"Computo",
  stock:{$lt:20}
})
```

5.3 Buscar productos de categoría "Audio" o "Accesorios" cuyo precio sea menor a \$1,500

```
db.productos.find({
  categoria:{$in:["Audio","Accesorios"]},
  precio:{$lt:1500}
})
```

5.4 Buscar productos que contengan la etiqueta "gaming" y tengan stock mayor a 10

```
db.productos.find({
  etiquetas:"gaming",
  stock:{$gt:10}
})
```

5.5 Buscar productos de las categorías "Audio" o "Computo", con precio mayor a \$1,000 y stock mayor a 15

```
db.productos.find({
  categoria:{$in:["Audio","Computo"]},
  precio:{$gt:1000},
  stock:{$gt:15}
})
```



Ejercicios de la Práctica 3

1. Consultar los productos cuya categoría no sea "Accesorios" ni "Almacenamiento".
2. Buscar productos cuya categoría sea "Accesorios" o "Almacenamiento".
3. Buscar productos cuyo stock sea mayor a 10 y menor a 30.
4. Consultar productos cuyo stock sea menor o igual a 25.
5. Consultar productos cuyo precio sea menor a \$1,000.
6. Consultar productos cuyo stock sea mayor o igual a 40.
7. Consultar productos cuyo stock sea mayor a 30.
8. Consultar todos los productos cuyo precio sea diferente de \$980.
9. Buscar todos los productos cuyo stock sea exactamente igual a 30.
10. Mostrar productos cuyo precio sea exactamente \$890.
11. Mostrar productos cuyo stock sea diferente de 25.
12. Consultar productos cuyo precio sea mayor a \$5,000.
13. Consultar productos cuyo precio sea mayor o igual a \$1,450.
14. Consultar productos cuyo stock sea menor a 30.
15. Mostrar productos cuyo precio esté entre \$500 y \$2,000, incluyendo ambos extremos.
16. Mostrar productos cuyo precio sea mayor a \$500 y menor a \$2,000, sin incluir los extremos.
17. Mostrar productos pertenecientes a "Audio" o "Accesorios".
18. Mostrar productos que no pertenezcan a "Audio" ni "Accesorios".
19. Consultar productos de categoría "Audio" cuyo precio sea mayor a \$1,000.
20. Consultar productos que sean de categoría "Computo" o tengan stock menor a 15 utilizando \$or.
21. Buscar productos que no sean de categoría "Audio" ni tengan precio mayor a \$5,000 utilizando \$nor.
22. Consultar los documentos que contienen el campo especificaciones.
23. Consultar los documentos que no contienen el campo especificaciones.
24. Consultar los productos que contienen la etiqueta "gaming".
25. Consultar los productos que contienen simultáneamente las etiquetas "audio" y "promocion" utilizando \$all.
26. Consultar los productos cuyo array etiquetas tenga exactamente dos elementos utilizando \$size.
27. Consultar productos de categoría "Accesorios" cuyo stock sea mayor a 30.
28. Consultar productos cuyo precio sea menor a \$2,000 y que contengan la etiqueta "usb".
29. Consultar productos de las categorías "Audio" o "Computo" que tengan stock mayor o igual a 20.
30. Diseñar una consulta propia que combine al menos tres condiciones y explicar qué información comercial obtiene.



PRÁCTICA 4. Proyección, ordenamiento y paginación

Objetivo

Controlar campos devueltos, orden, Top-N y páginas de resultados.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Ejercicios de salida

Practicar proyección, ranking y paginación.

```
db.productos.find({}, {_id:0,sku:1,nombre:1,precio:1}).sort({precio:-1}).limit(5)
db.productos.find({},
{_id:0,sku:1,nombre:1,stock:1}).sort({stock:1}).limit(10)
db.productos.find().sort({sku:1}).skip(0).limit(20)
db.productos.find().sort({sku:1}).skip(20).limit(20)
db.productos.find({categoria:"Computo"},{_id:0,sku:1,nombre:1,precio:1}).sort({precio:-1}).skip(0).limit(10)
```



PRÁCTICA 5. Modelado documental y relaciones

Objetivo

Comparar embedding y referencing y construir entidades relacionadas.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Ejercicio 1. Clientes y ventas

Crear colecciones relacionadas.

```
db.clientes.insertMany([
  {cliente_id:"C001",nombre:"Ana",ciudad:"Culiacan",segmento:"Frecuente"},
  {cliente_id:"C002",nombre:"Luis",ciudad:"Mazatlan",segmento:"Nuevo"},
  {cliente_id:"C003",nombre:"Marta",ciudad:"Los Mochis",segmento:"Frecuente"}
])
db.ventas.insertMany([
  {venta_id:"V001",cliente_id:"C001",fecha:ISODate("2026-01-15"),items:[
    {sku:"P001",cantidad:1,precio:17990},{sku:"P002",cantidad:2,precio:450}],total:18890},
  {venta_id:"V002",cliente_id:"C002",fecha:ISODate("2026-01-16"),items:[
    {sku:"P003",cantidad:1,precio:1250}],total:1250}
])
```

Ejercicio 2. Embedding vs referencing

Observar que items se embeben y cliente_id referencia otra colección.

```
db.ventas.find({cliente_id:"C001"})
db.ventas.find({"items.sku":"P001"})
```



PRÁCTICA 6. Aggregation Pipeline I

Objetivo

Usar \$match, \$group, acumuladores y \$sort para indicadores.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Pipeline básico

Calcular número de ventas, ingreso y ticket promedio.

```
db.ventas.aggregate([

{$group:{_id:null,ventas:{$sum:1},ingreso:{$sum:"$total"},ticket_promedio:{$
avg:"$total"}}}

])

db.ventas.aggregate([
{$group:{_id:"$cliente_id",compras:{$sum:1},gasto:{$sum:"$total"}}},
{$sort:{gasto:-1}}
])
```



PRÁCTICA 7. Aggregation Pipeline II

Objetivo

Usar \$project, expresiones, fechas, \$unwind y pipelines de varias etapas.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Pipeline avanzado

Descomponer items y analizar productos vendidos.

```
db.ventas.aggregate([
  {$unwind:"$items"},
  {$group:{_id:"$items.sku",unidades:{$sum:"$items.cantidad"},

ingreso:{$sum:{$multiply:["$items.cantidad","$items.precio"]}}}},
  {$sort:{ingreso:-1}},
  {$project:{_id:0,sku:"$_id",unidades:1,ingreso:1}}
])
db.ventas.aggregate([

{$project:{venta_id:1,anio:{$year:"$fecha"},mes:{$month:"$fecha"},total:1}},
  {$group:{_id:{anio:"$anio",mes:"$mes"},ingreso:{$sum:"$total"}}},
  {$sort:{"_id.anio":1,"_id.mes":1}}
])
```



PRÁCTICA 8. \$lookup y análisis relacional

Objetivo

Combinar colecciones relacionadas mediante \$lookup y \$unwind.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Cruce ventas-clientes

Aplicar \$lookup.

```
db.ventas.aggregate([  
  
  {$lookup:{from:"clientes",localField:"cliente_id",foreignField:"cliente_id",  
as:"cliente"}},  
  {$unwind:"$cliente"},  
  {$project:{_id:0,venta_id:1,fecha:1,total:1,  
              cliente:"$cliente.nombre",ciudad:"$cliente.ciudad"}},  
  {$sort:{total:-1}}  
])
```



PRÁCTICA 9. Índices y explain()

Objetivo

Medir consultas, crear índices simples/compuestos y comparar planes de ejecución.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Medición e índices

Comparar plan antes y después.

```
db.ventas.find({cliente_id:"C001"}).explain("executionStats")
db.ventas.createIndex({cliente_id:1})
db.ventas.find({cliente_id:"C001"}).explain("executionStats")
```

```
db.ventas.find({cliente_id:"C001", fecha:{$gte:ISODate("2026-01-01")}}).explain("executionStats")
db.ventas.createIndex({cliente_id:1, fecha:-1})
db.ventas.find({cliente_id:"C001", fecha:{$gte:ISODate("2026-01-01")}}).explain("executionStats")
```



PRÁCTICA 10. Python y PyMongo

Objetivo

Conectar Python, consultar, insertar y ejecutar pipelines desde PyMongo.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

PyMongo básico

Conectar, consultar e insertar desde Python.

```
from pymongo import MongoClient
from datetime import datetime

client = MongoClient("mongodb://localhost:27017/")
db = client["ventas_online"]

print(db.productos.find_one({"sku":"P001"}))

db.ventas.insert_one({
    "venta_id":"VPY001","cliente_id":"C003","fecha":datetime.now(),
    "items":[{"sku":"P005","cantidad":1,"precio":1600}],
    "total":1600
})

for doc in db.ventas.find({"total":{"$gte":1000}},{"_id":0}).limit(5):
    print(doc)

client.close()
```



PRÁCTICA 11. MongoDB y Pandas

Objetivo

Llevar resultados agregados a DataFrame para análisis tabular.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

MongoDB → Pandas

Agregar en MongoDB y transferir solo el resultado.

```
from pymongo import MongoClient
import pandas as pd

client = MongoClient("mongodb://localhost:27017/")
col = client.ventas_online.ventas
pipeline = [
    {"$group": {"_id": "$cliente_id", "ventas": {"$sum": 1},
               "gasto_total": {"$sum": "$total"},
               "ticket_promedio": {"$avg": "$total"}}},
    {"$sort": {"gasto_total": -1}}
]
df = pd.DataFrame(list(col.aggregate(pipeline)))
print(df.head())
print(df.describe(include="all"))
client.close()
```



PRÁCTICA 12. Generación masiva por lotes

Objetivo

Generar datos sintéticos reproducibles e insertarlos eficientemente por lotes.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Generador de 250 mil ventas

Insertar por lotes para evitar una lista gigante en memoria.

```
from pymongo import MongoClient
from random import choice, randint, uniform, seed
from datetime import datetime, timedelta
from time import perf_counter

seed(42)
client = MongoClient("mongodb://localhost:27017/")
col = client.ventas_online.ventas_masivas
col.drop()

clientes = [f"C{i:05d}" for i in range(1,5001)]
skus = [f"P{i:04d}" for i in range(1,501)]
inicio = datetime(2024,1,1)
N = 250_000
BATCH = 5000
lote = []
t0 = perf_counter()

for i in range(N):
    cantidad = randint(1,5)
    precio = round(uniform(100,25000),2)
    lote.append({
        "venta_id":f"VM{i:08d}",
        "cliente_id":choice(clientes),
```



```
"sku":choice(skus),
"fecha":inicio + timedelta(days=randint(0,730)),
"cantidad":cantidad,
"precio":precio,
"total":round(cantidad*precio,2),
"canal":choice(["Web","App","Marketplace"])
})
if len(lote) == BATCH:
    col.insert_many(lote, ordered=False)
    lote.clear()

if lote:
    col.insert_many(lote, ordered=False)

print("Documentos:", col.count_documents({}))
print("Segundos:", round(perf_counter()-t0,2))
client.close()
```



PRÁCTICA 13. Escalamiento progresivo

Objetivo

Medir el efecto del volumen sobre inserción, consulta y almacenamiento.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Experimento de escalamiento

Repetir con distintos tamaños y registrar tiempos.

Adaptar N a 100_000, 250_000, 500_000 y 1_000_000.

Para cada tamaño:

1) medir carga,

2) ejecutar 3 veces la misma consulta,

3) registrar mediana,

4) anotar stats de almacenamiento.

En mongosh:

```
db.ventas_masivas.stats()
```

```
db.ventas_masivas.find({canal:"App",total:{$gte:10000}}).explain("executionStats")
```



PRÁCTICA 14. Optimización y operaciones masivas

Objetivo

Comparar consultas con/sin índice y ejecutar updateMany/bulk_write.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Optimización masiva: Crear índice, clasificar y usar bulk_write.

```
db.ventas_masivas.find({canal:"App", fecha:{$gte:ISODate("2025-01-01")}}).explain("executionStats")
db.ventas_masivas.createIndex({canal:1, fecha:1, total:-1})
db.ventas_masivas.find({canal:"App", fecha:{$gte:ISODate("2025-01-01")}}).explain("executionStats")
```

```
db.ventas_masivas.updateMany(
  {total:{$gte:50000}},
  {$set:{segmento_venta:"alto_valor"}}
)
```

bulk_write desde Python

Actualizar lotes evitando update_one repetido.

```
from pymongo import MongoClient, UpdateOne

client = MongoClient("mongodb://localhost:27017/")
col = client.ventas_online.ventas_masivas

ops = []
for d in col.find({"segmento_venta":"alto_valor"}, {"_id":1}).limit(10000):
    ops.append(UpdateOne({"_id":d["_id"]}, {"$set":{"auditado":True}}))

if ops:
    r = col.bulk_write(ops, ordered=False)
    print("Modificados:", r.modified_count)
client.close()
```



PRÁCTICA 15. Proyecto integrador del dominio

Objetivo

Integrar modelado, análisis, optimización, Python y conclusiones reproducibles.

Actividades

- Ejecutar el código en el orden indicado y verificar el resultado.
- Registrar evidencia del antes y después cuando existan modificaciones.
- Interpretar el resultado en términos del problema de ventas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.

Proyecto final de Ventas

Resolver un miniestudio reproducible.

Requisitos mínimos:

- # - 5 indicadores de negocio.
- # - 1 análisis temporal.
- # - 1 ranking de clientes o productos.
- # - 2 pipelines de agregación.
- # - 1 consulta optimizada con explain() antes/después.
- # - 1 integración Python/Pandas.
- # - conclusión separando hallazgos de negocio y hallazgos de rendimiento.



BLOQUE II. BASE DE DATOS: HOSPITAL

Este bloque se resuelve como tarea. Debe transferirse el procedimiento aprendido en Ventas a un dominio diferente. Todos los datos son sintéticos y no se permite usar información clínica real.

Base de datos: hospital_db. Colecciones principales: pacientes, medicos y consultas.

PRÁCTICA 16. Primer contacto: base, colección y documentos

Nivel	Base	Volumen sugerido	Modalidad
Inicial	hospital_db	10–20	Tarea autónoma

Objetivo

Crear la base y la primera colección; insertar documentos con arrays y subdocumentos; reconocer _id y tipos BSON.

Ejercicio 1. Base y pacientes

Crear únicamente datos sintéticos.

```
use hospital_db
db.createCollection("pacientes")
db.pacientes.insertMany([
  {paciente_id:"PAC001",edad:45,municipio:"Culiacan",sexo:"F",
  alergias_simuladas:["penicilina"],contacto:{parentesco:"Familiar",telefono:"0000000000"}},
  {paciente_id:"PAC002",edad:31,municipio:"Mazatlan",sexo:"M",
  alergias_simuladas:[],contacto:{parentesco:"Familiar",telefono:"0000000000"}},
  {paciente_id:"PAC003",edad:67,municipio:"Navolato",sexo:"F",
  alergias_simuladas:["latex"],contacto:{parentesco:"Familiar",telefono:"0000000000"}},
])
db.pacientes.find()
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.



- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 17. CRUD y control de cambios

Nivel	Base	Volumen sugerido	Modalidad
Básico	hospital_db	50	Tarea autónoma

Objetivo

Aplicar inserción, lectura, actualización y eliminación con filtros seguros.

CRUD de pacientes

Resolver y documentar antes/después.

```
db.pacientes.insertOne({paciente_id:"PAC004",edad:52,municipio:"Guasave",sex  
o:"M",alergias_simuladas:[]})  
db.pacientes.find({municipio:"Culiacan"})  
db.pacientes.find({edad:{$gte:60}})  
db.pacientes.updateOne({paciente_id:"PAC002"},{$set:{municipio:"Escuinapa"}}  
)  
db.pacientes.updateMany({edad:{$gte:60}},{$set:{grupo_edad:"adulto_mayor"}})  
db.pacientes.find({paciente_id:"PAC004"})  
db.pacientes.deleteOne({paciente_id:"PAC004"})
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 18. Operadores, arrays y condiciones combinadas

Nivel	Base	Volumen sugerido	Modalidad
Básico-intermedio	hospital_db	100–500	Tarea autónoma



Objetivo

Resolver consultas con operadores de comparación, lógicos, existencia y arrays.

Banco de consultas

Explicar la necesidad de atención que resuelve cada consulta.

```
db.pacientes.find({edad:{$gte:40,$lte:65}})
db.pacientes.find({municipio:{$in:["Culiacan","Mazatlan"]}})
db.pacientes.find({municipio:{$nin:["Culiacan"]}})
db.pacientes.find({$and:[{edad:{$gte:60}}, {sexo:"F"}]})
db.pacientes.find({$or:[{alergias_simuladas:"penicilina"}, {alergias_simuladas:"latex"}]})
db.pacientes.find({alergias_simuladas:{$size:0}})
db.pacientes.find({grupo_edad:{$exists:true}})
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 19. Proyección, ordenamiento y paginación

Nivel	Base	Volumen sugerido	Modalidad
Intermedio	hospital_db	500–1,000	Tarea autónoma

Objetivo

Controlar campos devueltos, orden, Top-N y páginas de resultados.

Salida y paginación

Ordenar y devolver solo campos necesarios.

```
db.pacientes.find({}, {_id:0,paciente_id:1,edad:1,municipio:1}).sort({edad:-1}).limit(10)
db.pacientes.find({}, {_id:0,paciente_id:1,municipio:1}).sort({paciente_id:1}).skip(0).limit(20)
```



```
db.pacientes.find({},  
{_id:0,paciente_id:1,municipio:1}).sort({paciente_id:1}).skip(20).limit(20)
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



PRÁCTICA 20. Modelado documental y relaciones

Nivel	Base	Volumen sugerido	Modalidad
Intermedio	hospital_db	1,000–5,000	Tarea autónoma

Objetivo

Comparar embedding y referencing y construir entidades relacionadas.

Modelar médicos y consultas

Relacionar mediante identificadores.

```
db.medicos.insertMany([
  {medico_id:"MED001",especialidad:"Medicina interna",turno:"Matutino"},
  {medico_id:"MED002",especialidad:"Pediatria",turno:"Vespertino"},
  {medico_id:"MED003",especialidad:"Cardiologia",turno:"Matutino"}
])
db.consultas.insertMany([
  {consulta_id:"CON001",paciente_id:"PAC001",medico_id:"MED001",
    fecha:ISODate("2026-02-
01"),duracion_min:35,servicios_simulados:["valoracion"],costo_simulado:650},
  {consulta_id:"CON002",paciente_id:"PAC003",medico_id:"MED003",
    fecha:ISODate("2026-02-
02"),duracion_min:50,servicios_simulados:["valoracion","electrocardiograma"]
,costo_simulado:1200}
])
db.consultas.find()
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 21. Aggregation Pipeline I

Nivel	Base	Volumen sugerido	Modalidad
Intermedio	hospital_db	5,000–10,000	Tarea autónoma



Objetivo

Usar \$match, \$group, acumuladores y \$sort para indicadores.

Agregación de demanda

Calcular consultas y duración por médico.

```
db.consultas.aggregate([  
  
  {$group: {_id: "$medico_id", consultas: {$sum: 1}, duracion_promedio: {$avg: "$duracion_min"},  
    costo_total_simulado: {$sum: "$costo_simulado"}}},  
  {$sort: {consultas: -1}}  
])
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 22. Aggregation Pipeline II

Nivel	Base	Volumen sugerido	Modalidad
Intermedio-avanzado	hospital_db	10,000–50,000	Tarea autónoma

Objetivo

Usar \$project, expresiones, fechas, \$unwind y pipelines de varias etapas.

Análisis de servicios y meses

Usar \$unwind y fechas.

```
db.consultas.aggregate([  
  {$unwind: "$servicios_simulados"},  
  {$group: {_id: "$servicios_simulados", frecuencia: {$sum: 1}}},  
  {$sort: {frecuencia: -1}}  
])  
db.consultas.aggregate([
```



```
{ $project: { anio: { $year: "$fecha" }, mes: { $month: "$fecha" }, duracion_min: 1 } },
{ $group: { _id: { anio: "$anio", mes: "$mes" }, consultas: { $sum: 1 }, duracion_media: { $avg: "$duracion_min" } } },
{ $sort: { "_id.anio": 1, "_id.mes": 1 } }
])
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 23. \$lookup y análisis relacional

Nivel	Base	Volumen sugerido	Modalidad
Avanzado	hospital_db	10,000–50,000	Tarea autónoma

Objetivo

Combinar colecciones relacionadas mediante \$lookup y \$unwind.

\$lookup triple

Combinar consultas con pacientes y médicos.

```
db.consultas.aggregate([
{ $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "paciente_id", as: "paciente" } },
{ $unwind: "$paciente" },
{ $lookup: { from: "medicos", localField: "medico_id", foreignField: "medico_id", as: "medico" } },
{ $unwind: "$medico" },
{ $project: { _id: 0, consulta_id: 1, fecha: 1, duracion_min: 1,
municipio: "$paciente.municipio", especialidad: "$medico.especialidad" } }
])
```



Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



PRÁCTICA 24. Índices y explain()

Nivel	Base	Volumen sugerido	Modalidad
Avanzado	hospital_db	50,000–100,000	Tarea autónoma

Objetivo

Medir consultas, crear índices simples/compuestos y comparar planes de ejecución.

Índices clínicos sintéticos

Comparar nReturned, totalDocsExamined y executionTimeMillis.

```
db.consultas.find({medico_id:"MED001"}).explain("executionStats")
db.consultas.createIndex({medico_id:1})
db.consultas.find({medico_id:"MED001"}).explain("executionStats")

db.consultas.createIndex({medico_id:1,fecha:-1})
db.consultas.find({medico_id:"MED001",fecha:{$gte:ISODate("2026-01-01")}}).explain("executionStats")
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 25. Python y PyMongo

Nivel	Base	Volumen sugerido	Modalidad
Avanzado	hospital_db	100,000	Tarea autónoma

Objetivo

Conectar Python, consultar, insertar y ejecutar pipelines desde PyMongo.

PyMongo Hospital

Consultar datos sintéticos desde Python.

```
from pymongo import MongoClient
```



```
client = MongoClient("mongodb://localhost:27017/")
db = client["hospital_db"]

for d in
db.consultas.find({"duracion_min":{"$gte":30}},{"_id":0}).limit(10):
    print(d)

pipeline = [{"$group":{"_id":"$medico_id","consultas":{"$sum":1}}}]
print(list(db.consultas.aggregate(pipeline)))
client.close()
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 26. MongoDB y Pandas

Nivel	Base	Volumen sugerido	Modalidad
Avanzado	hospital_db	100,000–250,000	Tarea autónoma

Objetivo

Llevar resultados agregados a DataFrame para análisis tabular.

Hospital → Pandas

Extraer solo agregados.

```
from pymongo import MongoClient
import pandas as pd

client = MongoClient("mongodb://localhost:27017/")
col = client.hospital_db.consultas
pipeline = [
    {"$group":{"_id":"$medico_id","consultas":{"$sum":1},
        "duracion_promedio":{"$avg":"$duracion_min"},
        "costo_promedio":{"$avg":"$costo_simulado"}}},
```



```
{ "$sort": {"consultas": -1} }  
]  
df = pd.DataFrame(list(col.aggregate(pipeline)))  
print(df)  
client.close()
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



PRÁCTICA 27. Generación masiva por lotes

Nivel	Base	Volumen sugerido	Modalidad
Alto volumen	hospital_db	250,000	Tarea autónoma

Objetivo

Generar datos sintéticos reproducibles e insertarlos eficientemente por lotes.

Generador masivo de consultas sintéticas

Crear 250 mil atenciones ficticias.

```
from pymongo import MongoClient
from random import choice, randint, uniform, seed
from datetime import datetime, timedelta
from time import perf_counter

seed(42)
client = MongoClient("mongodb://localhost:27017/")
col = client.hospital_db.consultas_masivas
col.drop()

pacientes = [f"PAC{i:06d}" for i in range(1,50001)]
medicos = [f"MED{i:04d}" for i in range(1,501)]
especialidades = ["Medicina
interna","Pediatria","Cardiologia","Traumatologia","Dermatologia"]
inicio = datetime(2024,1,1)
N, BATCH = 250_000, 5000
lote, t0 = [], perf_counter()

for i in range(N):
    lote.append({
        "consulta_id":f"CM{i:08d}",
        "paciente_id":choice(pacientes),
        "medico_id":choice(medicos),
        "especialidad":choice(especialidades),
        "fecha":inicio + timedelta(days=randint(0,730)),
        "duracion_min":randint(10,90),
        "costo_simulado":round(uniform(300,2500),2),
        "urgencia_simulada":choice(["baja","media","alta"])
    })
    if len(lote)==BATCH:
        col.insert_many(lote, ordered=False); lote.clear()
if lote:
```



```
col.insert_many(lote, ordered=False)
```

```
print(col.count_documents({}), round(perf_counter()-t0,2))  
client.close()
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 28. Escalamiento progresivo

Nivel	Base	Volumen sugerido	Modalidad
Alto volumen	hospital_db	100k → 250k → 500k → 1M	Tarea autónoma

Objetivo

Medir el efecto del volumen sobre inserción, consulta y almacenamiento.

Escalamiento Hospital

Usar 100k, 250k, 500k y 1M.

```
# Repetir el generador cambiando N.  
# Registrar tiempo de carga y mediana de tres ejecuciones de:  
# especialidad + fecha + urgencia.  
db.consultas_masivas.stats()  
db.consultas_masivas.find({  
    especialidad:"Cardiologia",  
    fecha:{$gte:ISODate("2025-01-01")},  
    urgencia_simulada:"alta"  
}).explain("executionStats")
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.



- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 29. Optimización y operaciones masivas

Nivel	Base	Volumen sugerido	Modalidad
Alto volumen	hospital_db	1,000,000+	Tarea autónoma

Objetivo

Comparar consultas con/sin índice y ejecutar updateMany/bulk_write.

Optimización y clasificación masiva

Crear índice y clasificar consultas largas.

```
db.consultas_masivas.createIndex({especialidad:1,fecha:1,urgencia_simulada:1
})
db.consultas_masivas.find({
  especialidad:"Cardiologia",
  fecha:{$gte:ISODate("2025-01-01")},
  urgencia_simulada:"alta"
}).explain("executionStats")
```

```
db.consultas_masivas.updateMany(
  {duracion_min:{$gte:60}},
  {$set:{clasificacion_tiempo:"prolongada"}}
)
```

bulk_write

Actualizar una muestra desde Python.

```
from pymongo import MongoClient, UpdateOne
client = MongoClient("mongodb://localhost:27017/")
col = client.hospital_db.consultas_masivas
ops = [
  UpdateOne({"_id":d["_id"]}, {"$set":{"revisado_academico":True}})
  for d in
col.find({"clasificacion_tiempo":"prolongada"}, {"_id":1}).limit(10000)
]
```



```
if ops:
    r=col.bulk_write(ops,ordered=False)
    print(r.modified_count)
client.close()
```

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



PRÁCTICA 30. Proyecto integrador del dominio

Nivel	Base	Volumen sugerido	Modalidad
Integrador	hospital_db	Definido por el estudiante	Tarea autónoma

Objetivo

Integrar modelado, análisis, optimización, Python y conclusiones reproducibles.

Proyecto final Hospital

Resolver sin reutilizar nombres/consultas textuales del bloque Ventas.

Entrega mínima:
5 indicadores de demanda/operación.
1 análisis temporal.
1 ranking por especialidad o médico sintético.
2 pipelines.
1 \$lookup.
1 optimización con explain antes/después.
1 notebook Python/Pandas.
Datos exclusivamente sintéticos.

Actividades

- Reproducir la competencia utilizando datos completamente sintéticos.
- Agregar al menos dos consultas o variaciones propias sin copiar literalmente el bloque Ventas.
- Documentar resultados, errores encontrados y solución aplicada.
- Interpretar los resultados como indicadores operativos simulados, no como conclusiones clínicas.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



BLOQUE III. BASE DE DATOS: TURISMO

Este bloque funciona como examen práctico. Las consignas son equivalentes en competencia, pero el estudiante debe construir y justificar sus propias soluciones en el contexto turístico.

Base de datos: turismo_db. Colecciones principales: servicios, destinos y visitas.

PRÁCTICA 31. Primer contacto: base, colección y documentos

Nivel	Base	Volumen sugerido	Modalidad
Inicial	turismo_db	10–20	Examen práctico

Objetivo

Crear la base y la primera colección; insertar documentos con arrays y subdocumentos; reconocer _id y tipos BSON.

Base y servicios

Crear estructura inicial.

```
use turismo_db
db.createCollection("servicios")
db.servicios.insertMany([
  {servicio_id:"S001",destino:"Mazatlan",categoria:"Hotel",precio_noche:1800,
    amenidades:["wifi","alberca"],ubicacion:{zona:"Centro"},activo:true},
  {servicio_id:"S002",destino:"El Fuerte",categoria:"Tour",precio:950,
    amenidades:["guia","transporte"],ubicacion:{zona:"Centro
historico"},activo:true},
  {servicio_id:"S003",destino:"Cosala",categoria:"Cabaña",precio_noche:1450,
    amenidades:["estacionamiento","naturaleza"],activo:true}
])
db.servicios.find()
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



PRÁCTICA 32. CRUD y control de cambios

Nivel	Base	Volumen sugerido	Modalidad
Básico	turismo_db	50	Examen práctico

Objetivo

Aplicar inserción, lectura, actualización y eliminación con filtros seguros.

CRUD Turismo

Resolver operaciones equivalentes.

```
db.servicios.insertOne({servicio_id:"S004",destino:"Topolobampo",categoria:"Tour",precio:1200,amenidades:["lancha"]})
db.servicios.find({destino:"Mazatlan"})
db.servicios.find({precio_noche:{$gte:1000}})
db.servicios.updateOne({servicio_id:"S001"},{$set:{precio_noche:1950}})
db.servicios.updateMany({categoria:"Tour"},{$addToSet:{amenidades:"reservacion_online"}})
db.servicios.find({servicio_id:"S004"})
db.servicios.deleteOne({servicio_id:"S004"})
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 33. Operadores, arrays y condiciones combinadas

Nivel	Base	Volumen sugerido	Modalidad
Básico-intermedio	turismo_db	100-500	Examen práctico

Objetivo

Resolver consultas con operadores de comparación, lógicos, existencia y arrays.

Consultas de examen

Ejecutar y justificar; agregar 5 consultas propias.



```
db.servicios.find({precio_noche:{$gte:1000,$lte:2500}})
db.servicios.find({destino:{$in:["Mazatlan","Cosala"]}})
db.servicios.find({categoria:{$nin:["Hotel"]}})
db.servicios.find({$and:[{destino:"Mazatlan"},{amenidades:"wifi"}]})
db.servicios.find({$or:[{amenidades:"naturaleza"},{amenidades:"alberca"}]})
db.servicios.find({amenidades:{$all:["wifi","alberca"]}})
db.servicios.find({ubicacion:{$exists:true}})
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 34. Proyección, ordenamiento y paginación

Nivel	Base	Volumen sugerido	Modalidad
Intermedio	turismo_db	500–1,000	Examen práctico

Objetivo

Controlar campos devueltos, orden, Top-N y páginas de resultados.

Paginación de oferta

Crear Top-N y páginas.

```
db.servicios.find({},
{_id:0,servicio_id:1,destino:1,categoria:1,precio_noche:1}).sort({precio_noche:-1}).limit(10)
db.servicios.find().sort({servicio_id:1}).skip(0).limit(20)
db.servicios.find().sort({servicio_id:1}).skip(20).limit(20)
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.



Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



PRÁCTICA 35. Modelado documental y relaciones

Nivel	Base	Volumen sugerido	Modalidad
Intermedio	turismo_db	1,000–5,000	Examen práctico

Objetivo

Comparar embedding y referencing y construir entidades relacionadas.

Destinos y visitas

Diseñar relaciones.

```
db.destinos.insertMany([
  {destino_id:"D001",nombre:"Mazatlan",region:"Sur"},
  {destino_id:"D002",nombre:"El Fuerte",region:"Norte"},
  {destino_id:"D003",nombre:"Cosala",region:"Centro"}
])
db.visitas.insertMany([

{visita_id:"VIS001",destino_id:"D001",destino:"Mazatlan",fecha:ISODate("2026
-03-10"),

origen:"Sinaloa",noches:3,gasto_mxn:8500,tipo_viaje:"Vacaciones",personas:2,
  actividades:["playa","gastronomia"]},
  {visita_id:"VIS002",destino_id:"D002",destino:"El
Fuerte",fecha:ISODate("2026-03-12"),
  origen:"Sonora",noches:2,gasto_mxn:5200,tipo_viaje:"Cultural",personas:3,
  actividades:["cultura","gastronomia"]}
])
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



PRÁCTICA 36. Aggregation Pipeline I

Nivel	Base	Volumen sugerido	Modalidad
Intermedio	turismo_db	5,000–10,000	Examen práctico

Objetivo

Usar \$match, \$group, acumuladores y \$sort para indicadores.

Indicadores turísticos

Calcular demanda y gasto.

```
db.visitas.aggregate([
  {$group: {_id: "$destino", visitas: {$sum: 1}, personas: {$sum: "$personas"},
  gasto_total: {$sum: "$gasto_mxn"}, gasto_promedio: {$avg: "$gasto_mxn"}},
  {$sort: {gasto_total: -1}}
])
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 37. Aggregation Pipeline II

Nivel	Base	Volumen sugerido	Modalidad
Intermedio–avanzado	turismo_db	10,000–50,000	Examen práctico

Objetivo

Usar \$project, expresiones, fechas, \$unwind y pipelines de varias etapas.

Actividad y temporalidad

Analizar arrays y meses.

```
db.visitas.aggregate([
  {$unwind: "$actividades"},
```



```
{ $group: { _id: "$actividades", frecuencia: { $sum: 1 } } },
{ $sort: { frecuencia: -1 } }
])
db.visitas.aggregate([

{ $project: { anio: { $year: "$fecha" }, mes: { $month: "$fecha" }, personas: 1, gasto_mxn: 1 } },

{ $group: { _id: { anio: "$anio", mes: "$mes" }, personas: { $sum: "$personas" }, gasto: { $sum: "$gasto_mxn" } } },
{ $sort: { "_id.anio": 1, "_id.mes": 1 } }
])
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 38. \$lookup y análisis relacional

Nivel	Base	Volumen sugerido	Modalidad
Avanzado	turismo_db	10,000–50,000	Examen práctico

Objetivo

Combinar colecciones relacionadas mediante \$lookup y \$unwind.

Cruce con destinos

Aplicar \$lookup.

```
db.visitas.aggregate([

{ $lookup: { from: "destinos", localField: "destino_id", foreignField: "destino_id", as: "destino_info" } },
{ $unwind: "$destino_info" },
{ $project: { _id: 0, visita_id: 1, fecha: 1, gasto_mxn: 1, personas: 1,
destino: "$destino_info.nombre", region: "$destino_info.region" } }
])
```



])

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



PRÁCTICA 39. Índices y explain()

Nivel	Base	Volumen sugerido	Modalidad
Avanzado	turismo_db	50,000–100,000	Examen práctico

Objetivo

Medir consultas, crear índices simples/compuestos y comparar planes de ejecución.

Índices de examen

Comparar ejecución.

```
db.visitas.find({destino:"Mazatlan"}).explain("executionStats")
db.visitas.createIndex({destino:1})
db.visitas.find({destino:"Mazatlan"}).explain("executionStats")
```

```
db.visitas.createIndex({destino:1,fecha:-1,tipo_viaje:1})
db.visitas.find({destino:"Mazatlan",fecha:{$gte:ISODate("2026-01-01")},tipo_viaje:"Vacaciones"}).explain("executionStats")
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 40. Python y PyMongo

Nivel	Base	Volumen sugerido	Modalidad
Avanzado	turismo_db	100,000	Examen práctico

Objetivo

Conectar Python, consultar, insertar y ejecutar pipelines desde PyMongo.

PyMongo Turismo

El estudiante debe modificar filtro y proyección.

```
from pymongo import MongoClient
```



```
client = MongoClient("mongodb://localhost:27017/")
col = client.turismo_db.visitas

for d in col.find({"gasto_mxn":{"$gte":5000}},{"_id":0}).limit(10):
    print(d)

pipeline = [{"$group":{"_id":"$destino","gasto":{"$sum":"$gasto_mxn"}}},
            {"$sort":{"gasto":-1}}]
print(list(col.aggregate(pipeline)))
client.close()
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 41. MongoDB y Pandas

Nivel	Base	Volumen sugerido	Modalidad
Avanzado	turismo_db	100,000–250,000	Examen práctico

Objetivo

Llevar resultados agregados a DataFrame para análisis tabular.

Turismo → Pandas

Calcular indicadores por destino.

```
from pymongo import MongoClient
import pandas as pd

client = MongoClient("mongodb://localhost:27017/")
col = client.turismo_db.visitas
pipeline = [
    {"$group":{"_id":"$destino","visitas":{"$sum":1},
              "gasto_total":{"$sum":"$gasto_mxn"},
              "gasto_promedio":{"$avg":"$gasto_mxn"},
```



```
        "noches_promedio":{"$avg":"$noches"}}},
{"$sort":{"gasto_total":-1}}
]
df = pd.DataFrame(list(col.aggregate(pipeline)))
print(df)
client.close()
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



PRÁCTICA 42. Generación masiva por lotes

Nivel	Base	Volumen sugerido	Modalidad
Alto volumen	turismo_db	250,000	Examen práctico

Objetivo

Generar datos sintéticos reproducibles e insertarlos eficientemente por lotes.

Generador masivo Turismo

Crear 250 mil visitas sintéticas.

```
from pymongo import MongoClient
from faker import Faker
from random import choice, randint, uniform, seed
from datetime import datetime, timedelta
from time import perf_counter

seed(42)
fake = Faker("es_MX")
client = MongoClient("mongodb://localhost:27017/")
col = client.turismo_db.visitas_masivas
col.drop()

destinos = ["Mazatlan", "Culiacan", "El Fuerte", "Cosala", "Topolobampo"]
tipos = ["Negocios", "Vacaciones", "Cultural", "Naturaleza"]
inicio = datetime(2024, 1, 1)
N, BATCH = 250_000, 5000
lote, t0 = [], perf_counter()

for i in range(N):
    lote.append({
        "visita_id": f"V{i:08d}",
        "destino": choice(destinos),
        "fecha": inicio + timedelta(days=randint(0, 730)),
        "origen": fake.state(),
        "noches": randint(1, 10),
        "gasto_mxn": round(uniform(800, 30000), 2),
        "tipo_viaje": choice(tipos),
        "personas": randint(1, 6)
    })
    if len(lote) == BATCH:
        col.insert_many(lote, ordered=False); lote.clear()
if lote:
```



```
col.insert_many(lote,ordered=False)
```

```
print(col.count_documents({}), round(perf_counter()-t0,2))  
client.close()
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 43. Escalamiento progresivo

Nivel	Base	Volumen sugerido	Modalidad
Alto volumen	turismo_db	100k → 250k → 500k → 1M	Examen práctico

Objetivo

Medir el efecto del volumen sobre inserción, consulta y almacenamiento.

Escalamiento Turismo

Evaluar 100k → 1M.

```
# Repetir la generación con N = 100_000, 250_000, 500_000 y 1_000_000.  
# Registrar: tiempo de inserción, mediana de 3 consultas y stats.  
db.visitas_masivas.stats()  
db.visitas_masivas.find({  
    destino:"Mazatlan",  
    fecha:{$gte:ISODate("2025-01-01")},  
    gasto_mxn:{$gte:10000}  
}).explain("executionStats")
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.



Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.

PRÁCTICA 44. Optimización y operaciones masivas

Nivel	Base	Volumen sugerido	Modalidad
Alto volumen	turismo_db	1,000,000+	Examen práctico

Objetivo

Comparar consultas con/sin índice y ejecutar updateMany/bulk_write.

Optimización masiva Turismo

Crear índice y actualizar segmentos.

```
db.visitas_masivas.createIndex({destino:1,fecha:1,gasto_mxn:-1})
db.visitas_masivas.find({
    destino:"Mazatlan",
    fecha:{$gte:ISODate("2025-01-01")},
    gasto_mxn:{$gte:10000}
}).explain("executionStats")
```

```
db.visitas_masivas.updateMany(
    {gasto_mxn:{$gte:20000}},
    {$set:{segmento_gasto:"alto"}}
)
```

bulk_write de examen

Completar y reportar modified_count.

```
from pymongo import MongoClient, UpdateOne
client = MongoClient("mongodb://localhost:27017/")
col = client.turismo_db.visitas_masivas
ops = [
    UpdateOne({"_id":d["_id"]}, {"$set":{"validado_examen":True}})
    for d in col.find({"segmento_gasto":"alto"}, {"_id":1}).limit(10000)
]
if ops:
    resultado = col.bulk_write(ops, ordered=False)
    print("Modificados:", resultado.modified_count)
client.close()
```



Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



PRÁCTICA 45. Proyecto integrador del dominio

Nivel	Base	Volumen sugerido	Modalidad
Integrador	turismo_db	Definido por el estudiante	Examen práctico

Objetivo

Integrar modelado, análisis, optimización, Python y conclusiones reproducibles.

Proyecto/examen integrador Turismo

Resolver con evidencia y justificación propia.

```
# El examen debe incluir:  
# 1) cinco indicadores turísticos;  
# 2) análisis temporal mensual;  
# 3) ranking de destinos;  
# 4) análisis de gasto y estancia;  
# 5) dos pipelines con al menos cuatro etapas;  
# 6) un $lookup;  
# 7) explain() antes/después de un índice compuesto;  
# 8) procesamiento con PyMongo;  
# 9) DataFrame final en Pandas;  
# 10) conclusión técnica y sectorial.
```

Actividades

- Resolver la consigna de forma individual o según las reglas del examen.
- Modificar los ejemplos proporcionados para responder una pregunta turística diferente.
- Justificar cada decisión de modelado, consulta o índice.
- Entregar código ejecutable y evidencia del resultado obtenido.

Entregable

- Código completo y ejecutable.
- Capturas o evidencias relevantes.
- Resultados e interpretación.
- Respuesta a los retos de la práctica.
- Conclusión técnica individual.



ANEXO A. MATRIZ DE EQUIVALENCIA DE COMPETENCIAS

Competencia	Ventas	Hospital	Turismo
Primer contacto: base, colección y documentos	Laboratorio guiado	Tarea	Examen
CRUD y control de cambios	Laboratorio guiado	Tarea	Examen
Operadores, arrays y condiciones combinadas	Laboratorio guiado	Tarea	Examen
Proyección, ordenamiento y paginación	Laboratorio guiado	Tarea	Examen
Modelado documental y relaciones	Laboratorio guiado	Tarea	Examen
Aggregation Pipeline I	Laboratorio guiado	Tarea	Examen
Aggregation Pipeline II	Laboratorio guiado	Tarea	Examen
\$lookup y análisis relacional	Laboratorio guiado	Tarea	Examen
Índices y explain()	Laboratorio guiado	Tarea	Examen
Python y PyMongo	Laboratorio guiado	Tarea	Examen
MongoDB y Pandas	Laboratorio guiado	Tarea	Examen
Generación masiva por lotes	Laboratorio guiado	Tarea	Examen
Escalamiento progresivo	Laboratorio guiado	Tarea	Examen
Optimización y operaciones masivas	Laboratorio guiado	Tarea	Examen
Proyecto integrador del dominio	Laboratorio guiado	Tarea	Examen

ANEXO B. RÚBRICA GENERAL

Criterio	Ponderación
Funcionamiento técnico y corrección del código	30%
Evidencias y reproducibilidad	20%
Interpretación de resultados	25%
Resolución de ejercicios/retos	15%
Conclusión, orden y presentación	10%

ANEXO C. RECOMENDACIONES PARA PROCESAMIENTO MASIVO

- No generar un millón de documentos en una sola lista de Python; usar lotes de 1,000 a 10,000 documentos.
- Medir tiempos con `perf_counter()` y repetir consultas varias veces; reportar mediana cuando sea posible.



- Antes de comparar índices, mantener la misma consulta y un volumen comparable.
- No crear índices indiscriminadamente: aceleran lecturas específicas pero consumen almacenamiento y añaden costo de escritura.
- Usar Aggregation Pipeline para reducir datos en el servidor antes de convertir resultados a Pandas.
- En equipos con recursos limitados, reducir el volumen manteniendo la comparación relativa entre tamaños.
- Antes de updateMany() o deleteMany(), ejecutar el mismo filtro con find() y comprobar qué documentos serán afectados.
- Registrar las versiones de MongoDB, mongosh, Compass, Python y PyMongo empleadas.

ANEXO D. PLANTILLA DE TABLA DE RENDIMIENTO

Volumen	Índice	Consulta	Docs examinados	Docs devueltos	Tiempo (ms)	Observaciones
100k						
250k						
500k						
1M						
1M + índice						

ANEXO E. LISTA DE COMPROBACIÓN FINAL

- MongoDB Server, Compass y mongosh funcionan correctamente.
- Python accede a MongoDB mediante PyMongo.
- Cada dominio conserva sus propias colecciones y no mezcla datos de otro bloque.
- Los scripts de alto volumen usan inserción por lotes.
- Los experimentos de rendimiento incluyen explain('executionStats').
- Se comparan consultas antes y después de crear índices.
- Los DataFrames de Pandas se construyen preferentemente a partir de resultados agregados.
- El reporte diferencia ejecución técnica, interpretación y conclusiones.